



تقرير قدرة التحمل والأداء

منظومة وصلّي — فحص معماري عميق للسيرفر

تقييم قدرة النظام على تحمّل النمو المتوقّع خلال ٥ أشهر، وتحديد الاختناقات،
والإصلاحات المطبّقة، وخارطة الطريق للوصول للأرقام المستهدفة.

تم إجراء فحص معماري عميق للكود وبنية السيرفر وقاعدة البيانات ونظام الوقت الحقيقي والإشعارات. الفحص يكشف الاختناقات الحقيقية قبل حدوثها (وهو الأسلوب الآمن — بدل ضرب جمل حقيقي يوقع الخدمة على الزبائن الحاليين). النتيجة: النظام سليم ويعمل ممتاز للحجم الحالي، لكنه كان غير جاهز للأرقام المستهدفة بسبب نقاط محدّدة تم إصلاح أهمّها الآن، وبقيت نقاط معمارية تحتاج بنية تحتية إضافية.

الأرقام المستهدفة (خلال ٥ أشهر)

المؤشر	الرقم المستهدف
المطاعم	1,000
الزبائن	1,000,000
السائقون	300
ذروة الطلبات المتزامنة	~1,000 طلب/الثانية (ذروة)
رسائل الدعم	~1,000 رسالة/الثانية (ذروة)
التتبع الحيّ + الإشعارات	عشرات إلى مئات الآلاف من الاتصالات المتزامنة

الحكم النهائي بلمحة

الحالة	المعنى
✅ تم إصلاحه الآن	الفهارس، تخفيف حمل التتبع، البثّ الجماعي، سعة الاتصالات
⚠️ يحتاج بنية تحتية	توسيع أفقي للسيرفر + Redis للوقت الحقيقي + طابور للإشعارات + تخزين صور خارجي

تقرير وصلّي — قدرة التحمل

- سيرفر التطبيق: Node.js / Express — نسخة واحدة (Single Instance) على Railway.
- قاعدة البيانات: PostgreSQL (نسخة واحدة)، مجمّع اتصالات (Pool).
- الوقت الحقيقي: Socket.io لبتّ الطلبات وتتبع السائق — بدون **Redis Adapter**.
- الإشعارات: Firebase (أندرويد) + APNs (آيفون)، تُرسل من داخل السيرفر.
- الأداء: ضغط gzip مفعل، تخزين مؤقت جزئي، حدّ معدّل طلبات لكل IP (بالذاكرة).
- الصور: مخزّنة حاليًا كنص Base64 داخل قاعدة البيانات.

النقطة الجوهرية: البنية الحالية عمودية (نسخة واحدة). للوصول لمليون مستخدم لازم تصير أفقية (عدّة نسخ)، وهذا يتطلب إزالة الحالة المخزّنة بالذاكرة (Redis) — وهو محور التوصيات.

(1) غياب الفهارس (Indexes) حرج

الجدول الرئيسية (الطلبات، المستخدمين، المطاعم، الأصناف، الإشعارات) كانت بلا فهارس. أي استعلام كان يفحص الجدول كاملاً — يعني عند ملايين الصفوف تتحوّل الاستعلامات من أجزاء من الملي ثانية إلى ثواني، فيمتلئ مجمّع الاتصالات ويتوقّف السيرفر. هذا كان سيوقع النظام أول ما يكبر.

(2) الوقت الحقيقي بنسخة واحدة بلا Redis حرج للهدف

Socket.io على نسخة واحدة يتحمّل عملياً ~10,000–30,000 اتصال متزامن كحدّ أقصى، ولا يمكن توزيعه على عدّة نسخ بدون socket.io/redis-adapter@. عند مئات الآلاف من الاتصالات (تتبع + إشعارات) هذه النسخة الواحدة لن تكفي.

(3) الإشعار الجماعي لمليون مستخدم حرج للهدف

البتّ كان يرسل الإشعار لكل توكّن على حدة وبالتسلسل داخل نفس الطلب — إرسال مليون إشعار هكذا يستغرق ساعات ويفشل. يحتاج طابور خلفي (Queue) + إرسال FCM جماعي (Multicast).

(4) الصور Base64 داخل قاعدة البيانات مهم

صور المطاعم والأصناف والبنات مخزّنة كنص Base64 في قاعدة البيانات. هذا يضخّم حجم القاعدة ويبطئ الاستعلامات ويستهلك حزمة إنترنت هائلة مع كل تحميل منيو. الحل: تخزين خارجي (Object Storage) + CDN وحفظ الروابط فقط.

(5) كتابة موقع السائق على كل نبضة مهم

كان موقع كل سائق يُكتب في قاعدة البيانات مع كل إشارة (كل ٥ ثواني) + استعلام لإيجاد الزبون. يضاعف الحمل بلا داع.

(6) حالة مخزّنة بالذاكرة (حدّ المعدّل) مهم عند التوسّع

حدّ معدّل الطلبات مخزّن بذاكرة النسخة الواحدة — لن يعمل بشكل صحيح عند تشغيل عدّة نسخ. يجب نقله إلى Redis عند التوسّع الأفقي.

✓ إضافة كل الفهارس الناقصة **منفذ**

أضيفت فهارس شاملة على: الطلبات (حسب الزبون/المطعم/السائق/الحالة/التاريخ)، الأصناف، المستخدمين، الإشعارات، السائقين، التقييمات، العناوين، المحفظة، الدعم. **النتيجة:** الاستعلامات تبقى سريعة (ميلي ثانية) حتى مع ملايين الصفوف. هذا أكبر مكسب أداء منفرد.

✓ تخفيف حمل التتبع الحي **منفذ**

البت اللحظي للزبون يتم فورًا على كل نبضة (تجربة سلسلة)، بينما الكتابة لقاعدة البيانات صارت كل ١٥ ثانية فقط، مع كاش لهوية زبون كل طلب (إلغاء استعلام على كل نبضة). تقليل ضخ حمل قاعدة البيانات.

✓ البت الجماعي للإشعارات — إدراج مجمع **منفذ**

تخزين الإشعارات صار إدراجًا مجمعًا (Batch) بدل صف-صف — أسرع بمئات المرات عند الأعداد الكبيرة (مع إزالة التوكنات المكررة المطبقة سابقًا).

✓ رفع سعة اتصالات قاعدة البيانات **منفذ**

رفع الحد إلى ٣٠ اتصالًا وصار قابلاً للضبط عبر متغير بيئة حسب خطة Postgres.

✓ سكربت اختبار حمل (k6) **جاهز**

أرفق سكربت k6 جاهز لقياس الأداء فعليًا على بيئة اختبار (تصاعد حتى ٥٠٠ مستخدم متزامن) — backend/loadtest/k6- — wasaly.js.

النظام	قبل	بعد إصلاحات اليوم	للموصول لهدف الـ أشهر
قراءات قاعدة البيانات	فحص كامل — ينهار مبكرًا	مفهرس — ملايين الصفوف	خطة Postgres أقوى (4-8 vCPU)
كتابة الطلبات (ذروة 1000/ث)	محدود	أفضل	طابور/تجميع كتابة + Postgres قوي
سيرفر التطبيق	نسخة واحدة	نسخة واحدة	عدة نسخ + موازن حمل
الوقت الحقيقي (Sockets)	~10-30 ألف اتصال	نفسه	Redis Adapter + عدة نسخ
التتبع الحيّ (نبضات)	كتابة كل نبضة	كل 15 ث + كاش	يتوسّع مع الـ Sockets
إشعار جماعي (مليون)	تسلسلي — يفشل	DB مجعّ	طابور + FCM Multicast
الصور	Base64 بالقاعدة	نفسه	تخزين خارجي + CDN
حدّ المعدّل	بالذاكرة	نفسه	Redis عند تعدّد النسخ
الدعم الفني (1000/ث)	—	مفهرس	مراقبة تضخّم إشعار الإدمن

الخلاصة: بعد إصلاحات اليوم، قاعدة البيانات والتتبع صاروا جاهزين للنموّ الكبير. أما التوسّع الأفقي (عدة نسخ + Redis) والإشعار الجماعي وتخزين الصور فهي المتطلّبات المتبقّية للوصول لمليون مستخدم.

أولوية عالية (قبل الوصول لأرقام كبيرة)

- **@socket.io/redis-adapter + Redis**: لتوزيع الوقت الحقيقي على عدّة نسخ — هذا يفكّ سقف الاتصالات المتزامنة (التتبع + الإشعارات اللحظية).
- تشغيل عدّة نسخ من السيرفر خلف موازن جمل (**Autoscaling**): لتحمل ذروة الطلبات والزيارات.
- طابور إشعارات (**BullMQ** على **Redis**) + إرسال **FCM** جماعي: لإرسال ملايين الإشعارات بأمان وبالخلفية.
- نقل الصور لتخزين خارجي (**Cloudflare R2 / S3**) + **CDN**: يقلّص حجم القاعدة واستهلاك الإنترنت بشكل جذري ويسرّع كل الشاشات.

أولوية متوسطة

- ترقية خطة (**Postgres** CPU/RAM) + مراقبة الاستعلامات البطيئة، وإضافة نسخة قراءة (**Read Replica**) عند اشتداد القراءات.
- نقل حدّ المعدّل إلى **Redis** عند تعدّد النسخ.
- تجميع كتابة عناصر الطلب في إدراج واحد، وطابور لكتابة الطلبات وقت الذروة.
- مراقبة (**Monitoring**): **APM** + تنبيهات (**CPU**، الذاكرة، زمن الاستجابة، أخطاء).

ملاحظة صريحة ومهنية

إصلاحات اليوم (الفهارس + تخفيف التتبع + تجميع الإشعارات + سعة الاتصالات) رفعت سقف التحمل عشرات المرات وهي مطبّقة بالكود. لكن الوصول الكامل لمليون مستخدم و1000 طلب/ثانية يتطلّب البنود عالية الأولوية أعلاه — وهي قرارات بنية تحتية (**Redis** + عدّة نسخ + تخزين صور)، نفّذها على مراحل مع نموّ الأرقام الفعلي، فلا يتفاجأ المشروع في الشهر الخامس.

تقرير وصلّي — قدرة التحمل والأداء · نهاية التقرير